# UNIVERSIDAD DE EL SALVADOR

Facultad Multidisciplinaria De Occidente

Departamento de Ingeniería y Arquitectura

Ingeniería en Desarrollo de Software

**Asignatura:**

**Desarrollo y Reutilización de Software**

Ciclo X / Quinto año

**Tema:**

Sistema de Gestión de Inventarios con Componentes Reutilizables

**Coordinador de Cátedra:**

Ing. Dinora Virginia Osorio de Melara

**Tutor GT01:**

Ing. Dinora Virginia Osorio de Melara

**Estudiantes:**

Guillermo Alexander Rodriguez Cortez - RC22051

Salvador Isaías Juárez Alcántara - JA22005

Luis Eduardo Guiroa Linares - GL12016

Erick Giovanni Monroy López - ML22048

José Raúl Moreno Mejía - MM03168

Vilma Melissa Alvarado Parada - AP22024

Leonel Antonio Hernández Pérez - HP12002

**Fecha:**

25/10/2026

Ingeniería en DESARROLLO DE SOFTWARE

# Índice

**Introducción** ..... **3**

**Enfoque y Patrón de Diseño** ..... **4**

**Arquitectura y componentes** ..... **5**

**Flujo de interacción** ..... **6**

Versión de consola ..... 6

Versión con interfaz (GUI) ..... 6

Reportes ..... 6

Persistencia ..... 6

**Decisiones de diseño y reutilización** ..... **7**

**Conclusiones** ..... **8**

**Anexos** ..... **9**

Anexo 1. Enlace a repositorio de GitHub ..... 9

Anexo 2. Tabla de arquitectura y componentes ..... 9

Anexo 3. Formato del archivo inventario.json ..... 9

## Introducción

El presente documento describe el desarrollo de un Sistema de Gestión de Inventarios con Componentes Reutilizables, correspondiente al Tema 1. El proyecto consiste en una aplicación que permite agregar, eliminar y modificar productos, con la lógica dividida en componentes reutilizables.

En las siguientes secciones se explica cómo se aplicó el Patrón de Composición para organizar el sistema, cómo se estructuraron los componentes que lo conforman y cómo interactúan los dos clientes que consumen la misma lógica de negocio. También se detallan las funcionalidades implementadas, la forma en que se maneja la persistencia de los datos y las decisiones de diseño que hicieron posible reutilizar el backend en la aplicación de consola y en la interfaz gráfica.

## Enfoque y Patrón de Diseño

El diseño del sistema se apoyó en el Patrón de Composición, un enfoque de la programación orientada a objetos en el que las clases complejas se construyen a partir de otras más simples mediante relaciones de pertenencia (has-a), en lugar de recurrir a la herencia (is-a). La idea central es que un objeto pueda estar formado por otros objetos y delegar en ellos las tareas que les corresponden.

En este proyecto, esa clase contenedora es *Tienda*. En lugar de concentrar toda la lógica en un solo lugar, *Tienda* está compuesta por un *GestorDeInventario* y un servicio de *Reportes*. Cuando se le pide realizar una operación de inventario o generar un reporte, *Tienda* no la ejecuta directamente: se la encarga al componente correspondiente. Así, cada pieza conserva su propia responsabilidad y la tienda actúa como un coordinador entre ellas.

```
var tienda = new Tienda(new GestorDeInventario(), new Reportes());
```

Esta forma de organización tiene una consecuencia práctica importante: tanto la aplicación de consola como la interfaz gráfica trabajan únicamente con *Tienda*. Ninguno de los dos clientes necesita crear ni manipular directamente el gestor de inventario, ya que la tienda se encarga de ello internamente. De este modo, la lógica de negocio queda en un solo punto y los clientes se limitan a consumirla.

## Arquitectura y componentes

El sistema se organizó en componentes con responsabilidades bien definidas, de manera que cada uno cumple una función específica y se relaciona con los demás de forma clara. A continuación se describen los componentes que conforman la arquitectura:

- ***Modelos/Producto.cs*** define la estructura de datos de un producto, con sus campos *Id*, *Nombre*, *Precio* y *Cantidad*. Es utilizado tanto por la consola como por la GUI.
- ***Gestores/GestorDeInventario.cs*** encapsula la colección *List<Producto>* y las operaciones de agregar, eliminar, modificar y actualizar productos. Es utilizado internamente por *Tienda*.
- ***Servicios/Reportes.cs*** calcula las métricas del inventario y conserva la impresión del reporte en consola. Al igual que el gestor, es utilizado por *Tienda*.
- ***Tienda.cs*** es el contenedor que compone *GestorDeInventario* y *Reportes*, y expone operaciones de alto nivel. Es consumido por la consola y la GUI.
- ***MenuInventario.cs*** actúa como adaptador de consola: muestra el menú y delega cada acción en *Tienda*. *Program.cs* es el composition root de la consola, pues crea los componentes y arranca el menú.

En la GUI, *MainWindowViewModel.cs* es el adaptador que usa Tienda, prepara los datos para las vistas y coordina la persistencia. *InventarioStore.cs* se encarga de leer y escribir el inventario en el *inventario.json* de la raíz, y *MainWindow.axaml* define la ventana principal, la pestaña de inventario y la pestaña de reportes.

En conjunto, esta organización refleja la separación de responsabilidades: los modelos, la lógica de inventario, los reportes y la presentación residen en componentes distintos, lo que facilita el mantenimiento y la reutilización.

## Flujo de interacción

### Versión de consola

El programa principal *Program.Main* crea el gestor de inventario y el de reportes, los pasa a la tienda, y el menú recibe la tienda para mostrar las opciones. Cuando eliges algo en el menú, la tienda se encarga y reparte el trabajo entre sus componentes.

### Versión con interfaz (GUI)

La ventana principal *MainWindow* crea *MainWindowViewModel*, que a su vez crea la tienda y el almacén de datos I*nventarioStore*. Lo que haces en la ventana se manda a la tienda, que lo reparte entre el gestor y los reportes. Los resultados se acomodan para que la ventana los pueda mostrar.

### Reportes

Hay dos formas de generarlos. Una imprime el reporte directo en la consola *GenerarReporteInventario*, y la otra devuelve un objeto con los datos del reporte *CalcularReporte*: totales, producto más caro y más barato, y productos con poco stock. La ventana usa esta segunda forma para no repetir la misma lógica.

### Persistencia

La ventana guarda y carga todo en un archivo llamado *inventario.json* en la raíz del proyecto. La consola, en cambio, trabaja solo en memoria.

## Decisiones de diseño y reutilización

- **Composición sobre herencia:** Tienda no hereda de nada; agrupa y delega el trabajo en *GestorDeInventario* y *Reportes.*
- **Separación de responsabilidades:** Los modelos, la lógica de inventario, los reportes y la interfaz están en componentes distintos. Cada uno tiene su función y no invade la del otro.
- **Reutilización real:** La GUI no copia el código de la consola; enlaza los archivos del backend desde su *.csproj* con directivas Compile Include. Así ambas versiones comparten el mismo código fuente.
- **Sin duplicación de reportes:** La GUI no recalcula las métricas por su cuenta. Las obtiene mediante Tienda.GenerarReporte(), que delega en Reportes.CalcularReporte(...). Una sola lógica para ambos lados.
- **Adaptadores de interfaz:** *MenuInventario* y *MainWindowViewModel* adaptan la misma lógica de negocio a dos entornos: consola y GUI.
- **Persistencia separada:** *InventarioStore* se encarga únicamente del JSON, sin contaminar la lógica del inventario.

## Conclusiones

El desarrollo del sistema permitió comprobar en la práctica las ventajas del Patrón de Composición frente a la herencia en el diseño orientado a objetos. La clase *Tienda*, como contenedor que compone y delega en *GestorDeInventario* y *Reportes*, demostró que se puede alcanzar alta modularidad y separación de responsabilidades sin jerarquías rígidas de clases.

La reutilización del backend por dos clientes (consola y GUI utilizando *Avalonia*) es evidencia clara del enfoque: al enlazar los mismos archivos fuente desde el proyecto de la GUI se evitó duplicar lógica y ambas interfaces compartieron idénticas reglas de negocio, incluidos reportes y validaciones. También destacó el papel de los adaptadores de interfaz *MenuInventario* y *MainWindowViewModel* y de la persistencia separada *InventarioStore*.

En síntesis, el sistema cumple los objetivos: implementa el Patrón de Composición en C#, reutiliza componentes entre dos clientes, mantiene separación de responsabilidades y ofrece gestión de inventarios y reportes completa.

# Anexos

## Anexo 1. Enlace a repositorio de GitHub

Sistema de Gestión de Inventarios

## Anexo 2. Tabla de arquitectura y componentes

| Componente | Responsabilidad | Usado por |
|---|---|---|
| `Modelos/Producto.cs` | Define los datos de un producto: `Id`, `Nombre`, `Precio` y `Cantidad`. | Consola y GUI |
| `Gestores/GestorDeInventario.cs` | Encapsula la colección `List<Producto>` y las operaciones de agregar, eliminar, modificar y actualizar. | `Tienda` |
| `Servicios/Reportes.cs` | Calcula las métricas del inventario y conserva la impresión del reporte en consola. | `Tienda` |
| `Tienda.cs` | Contenedor que compone `GestorDeInventario` y `Reportes`; expone operaciones de alto nivel. | Consola y GUI |
| `MenuInventario.cs` | Adaptador de consola: muestra el menú y delega cada acción en `Tienda`. | Consola |
| `Program.cs` | Composition root de la consola: crea los componentes y arranca el menú. | Consola |
| `Gui/ViewModels/MainWindowViewModel.cs` | Adaptador de la GUI: usa `Tienda`, prepara los datos para las vistas y coordina la persistencia. | GUI |
| `Gui/Services/InventarioStore.cs` | Lee y escribe el inventario en el `inventario.json` de la raíz. | GUI |
| `Gui/Views/MainWindow.axaml` | Define la ventana principal, la pestaña de inventario y la pestaña de reportes. | GUI |

## Anexo 3. Formato del archivo inventario.json

```json
{
  "Id": 1,
  "Nombre": "Laptop HP",
  "Precio": 260,
  "Cantidad": 26
}
```